EXP.NO: 5	System Calls
DATE: 13/02/2026	

AIM:

To implement System calls fork() and exec().

ALGORITHM:

FORK:

1. **Start**
2. **Execute Call: The parent process invokes the fork() system call.**
3. **Kernel Action: The Operating System creates a new PCB (Process Control Block) and copies the parent's memory space to the child.**
4. **Return Value Check:**
5. **If Return Value == -1: Process creation failed (e.g., lack of memory).**
6. **If Return Value == 0: Execution is now within the Child Process.**
7. **If Return Value > 0: Execution is within the Parent Process (returns the Child's PID).**
8. **Divergent Paths: Use an if-else block to define different behaviors for the parent and child.**
9. **End**

EXEC:

1. **Start**
2. **Specify Path:** Provide the path of the executable file to be run (e.g., /bin/ls).
3. **Execute Call:** Invoke the exec() function (such as execl, execp, or execv).
4. **Memory Overlay:** The Operating System wipes the current code, data, and stack segments.
5. **Load New Program:** The new executable is loaded into the same address space.
6. **Start Execution:** The process begins running the new program from its main() function.
7. **Error Check:** If exec() returns, it means the call failed (e.g., file not found). If successful, the original program code is never reached again.
8. **End**
9.

CODE:

FORK

```
#include <stdio.h>
#include <unistd.h>

int main() {

   // child process execute the if block
   if(fork() == 0) {
      printf("This is CHILD process\n");
   }

   // parent process execute the else block
   else {
      printf("This is PARENT process\n");
   }

   printf("PID: %d\n", getpid());

   return 0;
}
```

EXEC

```
#include <stdio.h>
#include <unistd.h>

int main() {

   printf("Before exec()\n");

   execl("/bin/ls", "ls", NULL);

   printf("After exec()\n");

   return 0;
}
```

OUTPUT:

RESULT:

The implementation of system calls fork(0 and exec() are successful.